

Elevate

Sistema de gestión gastronómica — Tienda web + Punto de venta + ERP financiero

Documentación técnica de arquitectura

Next.js 16 · React 19 · TypeScript · PostgreSQL · Prisma

Documento generado para revisión técnica · Julio 2026

Índice

1. Por qué se construyó con Inteligencia Artificial
2. Visión general: qué es Elevate
3. Stack tecnológico y justificación
4. Arquitectura general (el "mapa mental")
5. Estructura de carpetas explicada y justificada
6. Dónde está el Frontend, el Backend y la Base de Datos
7. Cómo interactúan Frontend y Backend
8. Webhooks e integraciones externas
9. Ciberseguridad: capas, dónde y cómo
10. Flujos completos de ejemplo
11. Despliegue (deploy)
12. Riesgos detectados y recomendaciones

1. Por qué se construyó con Inteligencia Artificial

Este proyecto se desarrolló con asistencia de IA (agentes de codificación) siguiendo una metodología de **construcción por fases documentadas**. No fue "pedirle a la IA que haga la app" de una sola vez, sino un proceso dirigido donde cada módulo se especificó primero y se implementó después. La evidencia de esto está en el propio repositorio:

- `AGENTS.md` / `CLAUDE.md` — instrucciones maestras para la IA: reglas del proyecto, convenciones y advertencias (por ejemplo, "esta versión de Next.js tiene cambios que rompen compatibilidad, lee la documentación antes de escribir código").
- `actions/` — contiene **17 archivos de especificación por fase** (`FASE_1C_venta_fisica.md`, `FASE_2A_caja_layout...`, `FASE_3A_backend_finanzas.md`, ... `FASE_6B_hardening.md`, `FASE_6C_deploy.md`). Cada uno es el "contrato" de lo que la IA debía construir en esa etapa.
- `SECTION_SPECS.md`, `context.md`, `onboarding_fase6.md`, `pendientes.md` — documentos de contexto y seguimiento que mantienen la coherencia entre fases.

Ventajas de este enfoque

Beneficio	Cómo se logró
Velocidad	Módulos complejos (caja con turnos, contabilidad, inventario con recetas) se construyeron en días, no semanas.
Consistencia	Las especificaciones por fase forzaron patrones repetidos: mismo estilo de endpoint, misma validación, misma auditoría en todo el código.
Trazabilidad	Cada decisión de diseño quedó escrita en un <code>.md</code> , así que se puede auditar por qué existe cada pieza.
Mantenibilidad	Un desarrollador nuevo (humano o IA) puede leer las fases y entender el sistema sin ingeniería inversa.

Contracara honesta: el desarrollo asistido por IA a alta velocidad también deja "deudas" que hay que vigilar: configuración de despliegue peligrosa, drift entre ramas y secretos en archivos de entorno. Estos puntos se detallan en la sección 12.

2. Visión general: qué es Elevate

Elevate es una plataforma **todo-en-uno** para un negocio de comida. Reúne tres sistemas que normalmente son productos separados:

Sub-sistema	Para quién	Qué hace
Tienda / Menú web	Cliente final	Ver el catálogo por marca, armar pedido, elegir delivery o recojo.
Caja / POS	Cajero	Vender en mostrador (efectivo/QR), abrir y cerrar turno, gastos, fiados, entregar pedidos web.
Panel Admin / ERP	Dueño / Admin	Productos, insumos, recetas, inventario, contabilidad, flujo de caja, cuentas por cobrar/pagar, auditoría, usuarios, rastreo de repartidores.

Existe además un **envoltorio de escritorio** (Electron, `main.js`) que permite abrir la aplicación web como si fuera un programa nativo de Windows para la caja.

3. Stack tecnológico y justificación

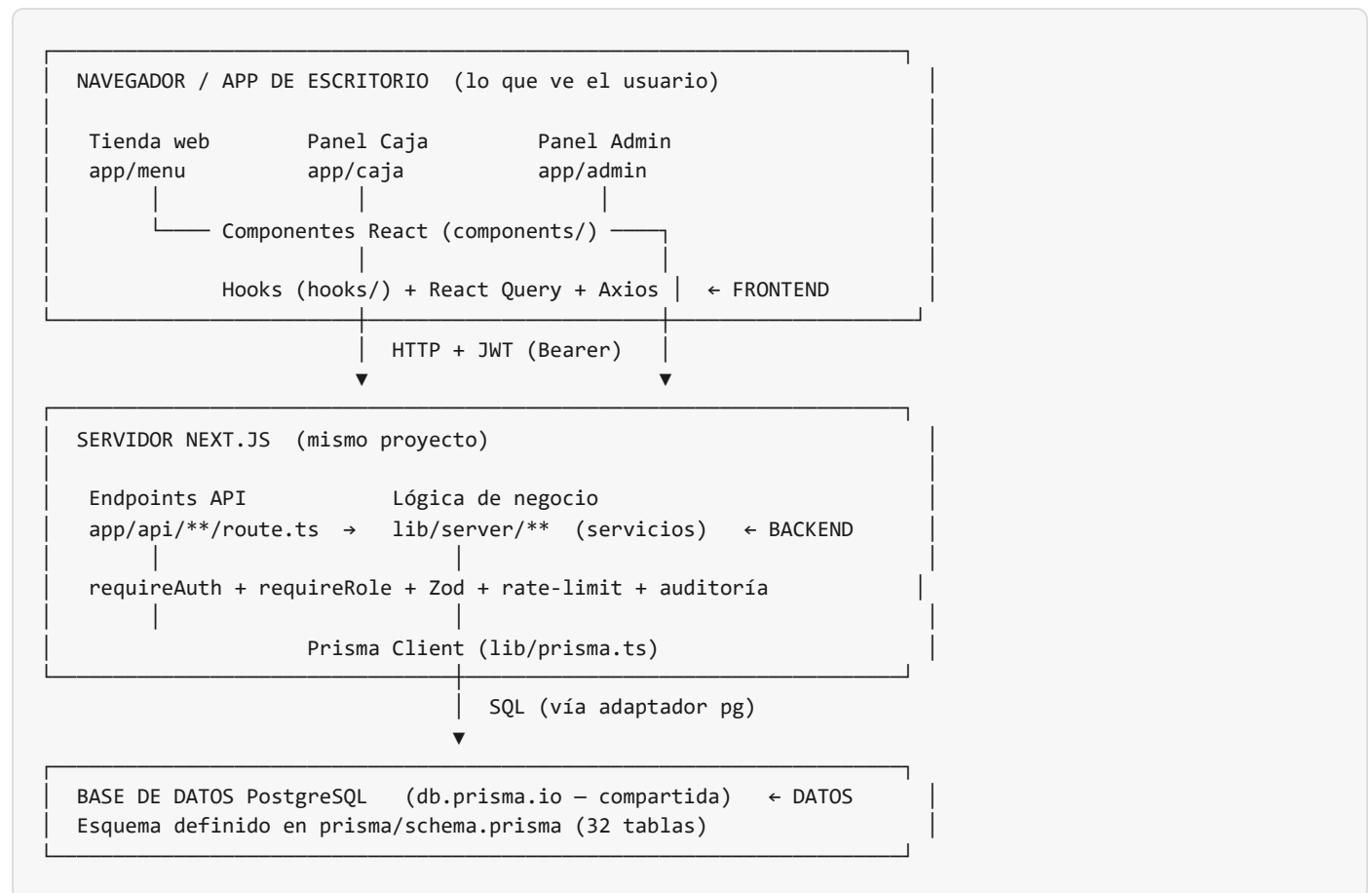
Capa	Tecnología	Por qué se eligió
Framework	Next.js 16 (App Router)	Un solo proyecto contiene frontend Y backend. Menos infraestructura, un solo despliegue, código compartido entre cliente y servidor.
UI	React 19 + TypeScript	Componentes reutilizables y tipado estático que atrapa errores antes de ejecutar.
Estilos	Tailwind CSS v4 + PrimeReact	Estilos rápidos y consistentes + componentes ricos ya hechos (tablas, diálogos).
Animación	Framer Motion	Transiciones fluidas en la tienda (experiencia "premium").
Datos (cliente)	TanStack React Query + Axios	Maneja peticiones al servidor, caché, refresco automático y estados de carga/error sin código repetitivo.
ORM	Prisma (+ adapter-pg)	Traduce entre el código TypeScript y las tablas SQL con tipado seguro. Evita escribir SQL a mano y previene inyección.
Base de datos	PostgreSQL	Base relacional robusta, ideal para datos financieros con integridad referencial (claves foráneas, transacciones ACID).
Auth	JWT + bcryptjs	Autenticación propia con control total sobre usuarios y roles en la base local.
Validación	Zod	Valida y limpia toda entrada del usuario en el servidor antes de tocar la base.
Mapas	Leaflet	Rastreo de repartidores en tiempo real.
Gráficos	Recharts	Estadísticas visuales del dashboard.
Archivos	Vercel Blob	Almacenar imágenes de productos fuera de la base de datos.
Escritorio	Electron	Empaquetar la web como app de Windows para la caja.

La decisión clave: un monolito full-stack

La justificación más importante de toda la arquitectura es que **frontend y backend viven en el mismo proyecto Next.js**. En lugar de tener un servidor de API separado (Node/Express) y una app de React aparte, Next.js permite que ambos convivan. Para un negocio de este tamaño esto significa: menos servidores que pagar y mantener, un solo despliegue, y tipos de datos compartidos entre lo que el navegador envía y lo que el servidor recibe.

4. Arquitectura general (el "mapa mental")

Todo el sistema sigue este flujo de tres capas. Léelo de arriba (usuario) hacia abajo (base de datos):



La regla de oro es **de una sola dirección**: el navegador nunca habla directo con la base de datos. Siempre pasa por un endpoint de API que verifica identidad, permisos y validez de los datos antes de dejar que Prisma toque PostgreSQL.

5. Estructura de carpetas explicada y justificada

Carpetas de aplicación

Carpeta	Rol	Qué contiene y por qué
app/	FRONT + BACK	El corazón de Next.js App Router. Cada subcarpeta es una <i>ruta</i> . Aquí conviven las páginas (frontend) y los endpoints (<code>app/api/</code> , backend). Se organiza así porque Next.js mapea carpetas a URLs automáticamente.
app/menu/ , app/login/ , app/driver/	FRONT	Páginas públicas: catálogo por marca, inicio de sesión, y vista del repartidor (accedida por token).
app/caja/	FRONT	Las 11 pantallas del cajero (venta, movimientos, apertura, cierre, gasto, deudores, historial, entregar, pedidos).
app/admin/	FRONT	Las ~23 pantallas del panel administrativo/ERP.
app/api/	BACK	Todo el backend HTTP. 81 endpoints (<code>route.ts</code>) agrupados por dominio: <code>auth</code> , <code>caja</code> , <code>pedidos</code> , <code>productos</code> , <code>admin/*</code> , etc.
components/	FRONT	Componentes React reutilizables, divididos por área: <code>admin/</code> , <code>caja/</code> , <code>shop/</code> (tienda) y <code>ui/</code> (piezas genéricas: botones, píldoras de método de pago, diálogos).
hooks/	FRONT	"Ganchos" de React que encapsulan la comunicación con el backend (ej. <code>caja.ts</code> , <code>finanzas.ts</code> , <code>auth.ts</code>). Un componente llama <code>useRegistrarVenta()</code> y no necesita saber cómo se hace la petición HTTP.
stores/	FRONT	Estado global compartido (categorías, reglas) para no repetir peticiones.
actions/	 Docs	Las especificaciones por fase de la construcción con IA (ver sección 1). No es código ejecutable.

Carpetas de servidor y datos

Carpeta	Rol	Qué contiene y por qué
lib/	BACK + FRONT	Utilidades compartidas. <code>lib/prisma.ts</code> (conexión a la BD), <code>lib/auth.ts</code> (JWT/bcrypt), <code>lib/Guard.tsx</code> (protección de rutas en el cliente), <code>lib/providers.tsx</code> (React Query).
lib/server/	BACK	La lógica de negocio real. Separada de los endpoints para poder reutilizarla y testearla. Subcarpetas por dominio: <code>auth/</code> (sesiones), <code>caja/</code> , <code>finanzas/</code> (contabilidad, flujo, gastos), <code>inventario/</code> (descuento de stock por recetas), <code>clientes/</code> , <code>audit/</code> (bitácora), <code>dto/</code> (esquemas Zod de validación).
prisma/	DB	<code>schema.prisma</code> — la definición de las 32 tablas y sus relaciones. <code>migrations/</code> — el historial de cambios de la base. <code>seed.ts</code> — datos iniciales.
public/	FRONT	Archivos estáticos servidos tal cual: logo, íconos, imágenes.
data/	Datos	Datos semilla o de apoyo.
scripts/	Ops	Utilidades de operación creadas para mantenimiento: respaldo (<code>db-backup.mjs</code>), limpieza (<code>db-clean.mjs</code>) y el SQL de reparación del módulo de fiado.
backups/	Ops	Respaldos JSON de la base (excluidos de git por contener datos de clientes).
docs/	Docs	Documentación: guía de usuario final, este documento, y el handoff de base de datos.
imagesExtra/	Ref	Material de referencia del ERP "Paladar" que inspiró módulos de POS/finanzas.

Archivos de configuración en la raíz

Archivo	Para qué sirve
package.json	Lista de dependencias y comandos (<code>dev</code> , <code>build</code> , <code>start</code>).
prisma.config.ts	Configuración de Prisma para comandos de línea (migraciones, studio).
next.config.ts	Ajustes de Next.js (imágenes, paquetes externos del servidor).
vercel.json	Instrucciones de despliegue en Vercel. ⚠️ Contiene un comando peligroso — ver sección 12.
docker-compose.yml	Levanta un PostgreSQL local en Docker para desarrollo.
main.js	Punto de entrada de la app de escritorio Electron.
.env	Secretos y configuración: URL de la base, secreto JWT. ⚠️ Nunca debe subirse a git.
tsconfig.json , eslint.config.mjs , postcss.config.mjs	Configuración de TypeScript, linter y procesamiento de CSS.

6. Dónde está el Frontend, el Backend y la Base de Datos

Esta es la pregunta más frecuente en un proyecto Next.js, porque todo vive junto. La separación es **por convención de carpetas**, no por proyectos distintos:

FRONTEND — lo que corre en el navegador

- `app/**/page.tsx` — las páginas visibles (llevan `'use client'` cuando son interactivas).
- `components/` — piezas visuales reutilizables.
- `hooks/` — comunicación con el backend desde el navegador.
- `stores/`, `lib/Guard.tsx`, `lib/providers.tsx` — estado y protección de rutas del lado cliente.

BACKEND — lo que corre en el servidor

- `app/api/**/route.ts` — los 81 endpoints HTTP. Es la *puerta* del backend.
- `lib/server/` — la *lógica* del backend (servicios de caja, finanzas, inventario, etc.).
- `lib/auth.ts`, `lib/server/auth/session.ts` — autenticación y sesiones.
- `lib/prisma.ts` — el único punto que habla con la base de datos.

BASE DE DATOS — dónde viven los datos

- **Motor:** PostgreSQL alojado en `db.prisma.io` (servicio gestionado, compartido entre desarrollo y producción).
- **Definición:** `prisma/schema.prisma` describe las 32 tablas (Usuario, Producto, Insumo, Transaccion, CajaTurno, MovimientoCaja, CuentaCorriente, RegistroAuditoria, etc.).
- **Acceso:** exclusivamente vía Prisma Client. Ningún componente del navegador puede tocarla directamente.

Analogía: el *frontend* es el salón del restaurante (lo que ve el cliente), el *backend* es la cocina (donde se procesan los pedidos con reglas y controles), y la *base de datos* es la despensa/archivo (donde se guarda todo). El mesero (API) es el único que cruza entre el salón y la cocina; el cliente nunca entra a la despensa.

7. Cómo interactúan Frontend y Backend

La comunicación es siempre por **HTTP**, con un **token JWT** que identifica al usuario. Este es el viaje completo de una acción típica ("el cajero registra una venta"):

- | | |
|--|--|
| 1. El cajero pulsa "Cobrar" en la pantalla | → <code>app/caja/venta/page.tsx</code> (FRONT) |
| 2. El componente llama a un hook | → <code>useRegistrarVenta()</code> en <code>hooks/caja.ts</code> |

3. El hook usa Axios para enviar POST /api/caja/venta
 - Axios añade el header Authorization: Bearer <token> (interceptor en hooks/api.ts)
4. Llega al endpoint → app/api/caja/venta/route.ts (BACK)
 - isRateLimited() → ¿demasiadas peticiones? (máx. 3 en 10s)
 - requireAuth() → verifica el JWT y carga el usuario activo
 - requireRole() → ¿es CAJERO/ADMIN/DUENO?
 - VentaFisicaDTO.parse → valida los datos con Zod
5. El endpoint delega en la lógica de negocio → lib/server/caja/caja.service.ts
 - recalcula el total EN EL SERVIDOR (no confía en el precio del navegador)
 - descuenta stock por receta, registra el movimiento de caja, audita
 - todo dentro de una TRANSACCIÓN (o se hace todo, o no se hace nada)
6. Prisma traduce a SQL y escribe en PostgreSQL → lib/prisma.ts
7. La respuesta JSON regresa al navegador; React Query actualiza la pantalla

Las piezas que hacen esto posible

Pieza	Archivo	Función
Cliente HTTP	hooks/api.ts	Axios con dos "interceptores": uno añade el token a cada petición; otro, si recibe 401, borra el token y manda al login.
Caché de datos	lib/providers.tsx	React Query cachea respuestas, evita peticiones duplicadas y refresca datos automáticamente.
Guardia de rutas	lib/Guard.tsx	ProtectedRoute envuelve páginas privadas: comprueba el token y el rol antes de mostrarlas (protección de <i>experiencia</i> , no de seguridad — la real está en el backend).
Sesión de servidor	lib/server/auth/session.ts	requireAuth lee el token (del header o de una cookie), lo verifica y carga el usuario. requireRole aplica los permisos.

Detalle de seguridad importante: el precio y el total de una venta **siempre se recalculan en el servidor** a partir de los precios reales de la base. Aunque alguien manipule el navegador para "pagar" un producto a 0 Bs, el backend ignora ese dato y usa el precio verdadero. Nunca se confía en el cliente.

8. Webhooks e integraciones externas

El sistema es mayormente auto-contenido, pero tiene varios puntos de integración con el exterior:

8.1 Enlace del repartidor (patrón tipo webhook por token)

Cuando un pedido sale a delivery, el sistema genera un `driver_link_id` (un identificador corto e impredecible). El repartidor abre `app/driver/[token]` desde su celular, sin necesidad de tener cuenta. Ese token le da acceso *solo* a ese pedido, y le permite:

- Ver los datos del pedido: `GET /api/pedidos/driver/[token]`
- Actualizar el estado (EN_CAMINO → LLEGÓ → ENTREGADO): `POST` del mismo endpoint
- Enviar su ubicación GPS en vivo (`driver_lat` , `driver_lng`), que el admin ve en el mapa Leaflet.

Es un patrón "webhook-like": una URL con un token secreto reemplaza al login para un actor externo de un solo uso.

8.2 Almacenamiento de imágenes (Vercel Blob)

Las imágenes de productos no se guardan en la base de datos (sería lento y caro), sino en **Vercel Blob**, un almacenamiento de archivos en la nube. La base solo guarda la URL. El endpoint `app/api/admin/upload` gestiona la subida.

8.3 Alertas de inventario (WhatsApp — simulado/preparado)

Existe un servicio de alertas (`lib/server/alertas/whatsapp.service.ts`) que, cuando los insumos bajan del umbral crítico, prepara y registra notificaciones (por ahora en modo simulado, listo para conectar un proveedor real de WhatsApp). Los envíos quedan registrados en la tabla `RegistroAlerta` .

8.4 Sondeo en tiempo real (polling)

Para "tiempo real" sin la complejidad de WebSockets, el frontend usa **polling**: el hook `useOrderPolling.ts` vuelve a preguntar por pedidos nuevos cada pocos segundos. Así el panel de caja/admin muestra pedidos entrantes y el mapa de repartidores se actualiza sin recargar la página.

9. Ciberseguridad: capas, dónde y cómo

La seguridad está implementada en **capas** (defensa en profundidad). Cada petición sensible atraviesa varios controles antes de tocar la base de datos:

#	Control	Dónde	Cómo protege
1	Contraseñas cifradas	<code>lib/auth.ts</code>	bcrypt con "salt" (10 rondas). Las contraseñas nunca se guardan en texto plano; ni el administrador puede leerlas.
2	Tokens JWT firmados	<code>lib/auth.ts</code>	Al iniciar sesión se emite un token firmado con un secreto. Sin ese secreto nadie puede falsificar una identidad. Expira a las 20 horas.
3	Verificación por endpoint	<code>session.ts</code> → <code>requireAuth</code>	Cada endpoint privado revalida el token y confirma que el usuario existe y está <i>activo</i> en la base (un usuario desactivado pierde acceso al instante).
4	Control de roles (RBAC)	<code>requireRole</code>	Cada endpoint declara qué roles pueden usarlo (DUENO, ADMIN, CAJERO, CLIENTE). Un cajero no puede entrar a funciones de dueño.
5	Validación de entrada	<code>lib/server/dto/*</code> (Zod)	Todo dato entrante se valida contra un esquema estricto (tipos, rangos, longitudes máximas). Rechaza datos malformados o maliciosos antes de procesarlos.
6	Límite de peticiones	<code>rate-limit</code>	Frena abusos: p. ej. máximo 3 ventas en 10 segundos. Mitiga ataques de fuerza bruta y duplicados.
7	Recálculo en servidor	<code>caja.service.ts</code>	Precios y totales se calculan con datos de la base, nunca con lo que envía el navegador. Impide manipulación de precios.
8	Lista blanca de campos	<code>pedidos/[id]/route.ts</code>	Al actualizar un registro solo se aceptan campos explícitamente permitidos. Previene "mass assignment" (que alguien inyecte campos que no debería).
9	Bitácora de auditoría	<code>audit.service.ts</code>	Cada acción sensible (venta, apertura/cierre de caja, cambios) se registra con usuario, rol, IP y navegador en una tabla <i>solo-append</i> . Trazabilidad total.
10	Protección contra inyección SQL	Prisma	Prisma usa consultas parametrizadas por diseño. No se concatena SQL a mano, así que la inyección SQL clásica no aplica.
11	Aislamiento de paquetes de servidor	<code>next.config.ts</code>	bcrypt y Prisma se marcan como "externos del servidor": nunca se envían al navegador, evitando filtrar lógica sensible.

Cómo se ve en la práctica (un endpoint real)

```
export async function POST(req) {
  if (isRateLimited(req, 10000, 3)) // 6. límite de peticiones
    return 429;
  const session = await requireAuth(req); // 3. verifica identidad (JWT)
  requireRole(session, ['CAJERO', 'DUENO', 'ADMIN']); // 4. permisos por rol
  const dto = VentaFisicaDTO.parse(body); // 5. valida entrada (Zod)
  const venta = await caja.registrarVentaFisica(session, dto, {ip, userAgent});
  // ↑ dentro: recálculo en servidor (7) + auditoría con IP (9), todo en transacción
}
```

Fortaleza: la seguridad real vive en el *backend*. El guardia del frontend (`ProtectedRoute`) solo mejora la experiencia (no muestra pantallas que no corresponden), pero aunque alguien lo saltee, el backend rechaza cualquier petición sin token válido y rol correcto. Esto es lo correcto: *nunca confiar en el cliente*.

10. Flujos completos de ejemplo

10.1 Venta en caja (mostrador)

```
Cajero abre turno → agrega productos al carrito → elige EFECTIVO/QR → Cobrar
↓
POST /api/caja/venta → valida → recalcula total → crea Transaccion
↓
descuenta stock de insumos por receta (MovimientoInterno)
↓
registra MovimientoCaja + actualiza saldo de la cuenta (EFECTIVO o QR)
↓
audita la operación → responde → el dashboard refleja la venta al instante
```

10.2 Pedido web con delivery

```
Cliente arma pedido en la tienda → POST /api/pedidos (crea Transaccion PENDIENTE)
↓
El cajero lo ve por polling → lo prepara → genera enlace de repartidor (token)
↓
Repartidor abre app/driver/[token] → actualiza estado + envía GPS
↓
Admin ve la ubicación en el mapa (Leaflet) → ENTREGADO → se descuenta stock y se cierra
```

10.3 Cierre de caja

```
Cajero cuenta el dinero físico → POST /api/caja/cierre (real_efectivo, real_qr)
↓
El sistema calcula ESPERADO = apertura + ventas - gastos
↓
Compara ESPERADO vs REAL → registra la DIFERENCIA (sobrante/faltante) por método
↓
Marca el turno CERRADO y lo archiva en el historial (auditado)
```

11. Despliegue (deploy)

El proyecto está preparado para desplegarse en **Vercel** (la plataforma nativa de Next.js). El proceso, definido en `vercel.json`, hace: instalar dependencias → generar el cliente Prisma → sincronizar el esquema con la base → construir la app.

Alternativamente, la caja puede correr como **app de escritorio** empaquetada con Electron (`npm run electron:build`), que abre la web en una ventana nativa de Windows.

12. Riesgos detectados y recomendaciones

Durante la revisión se identificaron puntos que conviene atender. Se ordenan por severidad:

● CRÍTICO — `vercel.json` ejecuta `prisma db push --accept-data-loss` en cada build

El comando de despliegue incluye `prisma db push --accept-data-loss`, que fuerza la base a coincidir con el esquema y *acepta perder datos si hay diferencias*. Si se despliega desde una rama con el esquema desactualizado, **puede borrar tablas y datos de producción** — exactamente el incidente que ocurrió con el módulo de fiado. **Recomendación:** quitar `db push --accept-data-loss` del build y usar migraciones controladas (`prisma migrate deploy`), que nunca destruye datos.

● ALTO — Secretos en `.env`

El archivo `.env` contiene la URL de la base (con usuario y contraseña) y el secreto JWT en texto plano. Debe estar **siempre** en `.gitignore` y nunca subirse al repositorio. Si alguna vez se subió, esas credenciales deben rotarse (cambiarse). El secreto JWT actual es corto y predecible; conviene reemplazarlo por uno largo y aleatorio.

● MEDIO — JWT en `localStorage`

El token se guarda en `localStorage` del navegador, lo que lo hace accesible a JavaScript y por tanto vulnerable a ataques XSS. Una alternativa más segura es una cookie `httpOnly` (inaccesible desde JavaScript). El backend ya soporta leer el token desde cookie, así que la migración es viable.

● MEDIO — Token del repartidor corto

El `driver_link_id` es un fragmento de UUID (8 caracteres). Para un enlace de bajo valor y corta vida es aceptable, pero conviene usar un token más largo y con expiración para evitar que se adivine.

● MENOR — Base de datos compartida sin coordinación de migraciones

Desarrollo y producción usan la misma base, y distintas ramas han aplicado esquemas diferentes. Se recomienda separar la base de desarrollo de la de producción, y acordar que los cambios de esquema siempre pasen por migraciones revisadas (nunca `db push` directo).